

Based on the lab experiences and exam of [IoT](#).

The arguments are:

- LoRaWAN
- Cayenne LPP and API
- BLE & MQTT

1) LoRaWan

In the lab we worked with Arduino that via an antenna was able to connect to LoRaWan. The required libraries are: MKRWAN (not V2)

With the **first configuration** example we are able to print the device information. Each device must be registered in the network and the data that we need is:

Name	Meaning	Source
Dev EUI	ID of arduino	Arduino "first configuration"
Join/App EUI	Identifies server	Arbitrary HEX String
App Key	identifies app in TTN	Randomly generated during setup
End-Device-ID	Name of device	Can be chosen

There are two joining methods:

- Over The Air Activation (OTAA): We use this
- ABP

This link is used to enter TTN: <https://eu1.cloud.thethings.network/console>

In the "Live data" section the messages are visible. These must be decoded and a decoder can be added in the "Payload Formatter" section.

LoRaWAN Theory

Long Range Wireless Access Network (LoRaWAN) is a protocol developed in 2009 for IoT long distance communication. LoRa is the physical implementation, while LoRaWAN is the Rest of the protocol stacks.

Let's start with the PHY layer:

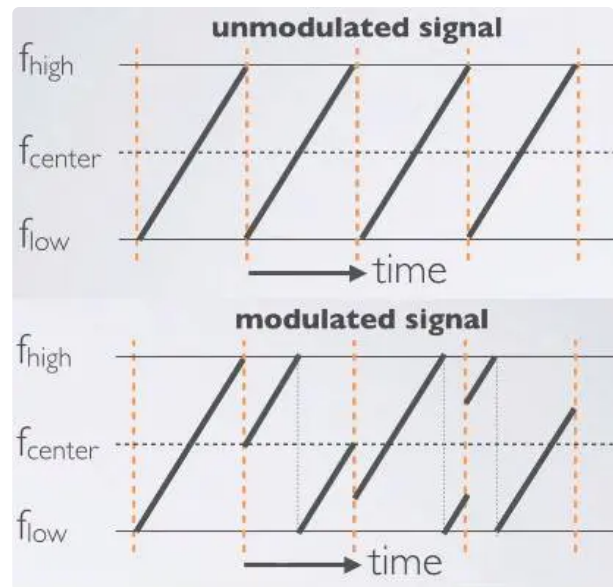
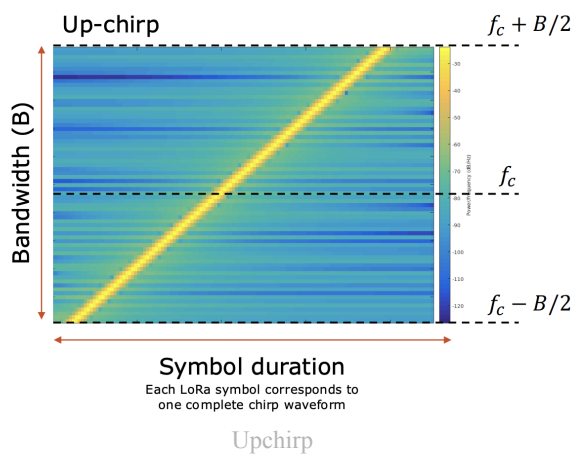
FREQUENCY RANGE

It uses lower frequencies (radio) in order to allow for higher distance communication but lower throughput. **Usually around 868 MHz.**

It uses **24-80 channels of 125 KHz each**. The minimum number of channels is 3 since they are required for the join request.

MODULATION

It uses the **Chirp SpreadSpectrum (CSS)**. Essentially Symbols are encoded by modulating a carrier that changes frequency linearly in time. It has the advantage to spread the power on a wider spectrum in order to resist noise. It is divided in up and down chirps. The slope is always the same, however based on where it starts (and then wraps around) encodes a different symbol. How many symbols are encoded? Look at [SF]/(#Spreading Factor)



Modulation Example

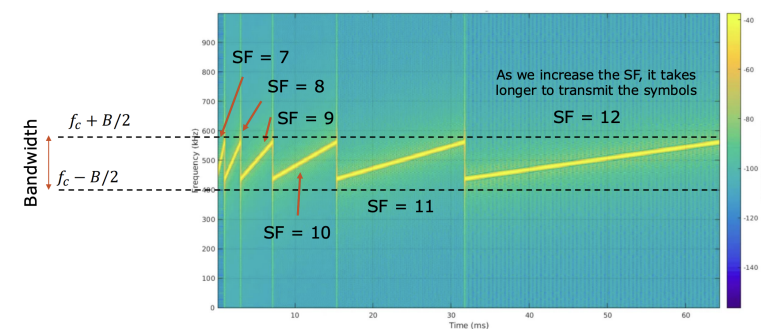
SPREADING FACTOR

The spreading factor **SF** is a LoRa-specific parameter that defines **how many chips encode one symbol**. Therefore each symbol is made of 2^{SF} chips. What this means is that a **chip is the resolution of the waveform**.

LoRa can assume SF values between 7 to 12 and this influences data rate, time on air, battery life and receiver sensitivity

The chip rate is fixed (resolution constant) and equal to the bandwidth, however, clearly, the symbol time changes linearly with the chips.

Symbol Time	Symbol Rate	Bitrate	Energy Consumption
$T_S = \frac{2^{SF}}{B}$	$R_S = \frac{1}{T_S} = \frac{B}{2^{SF}}$	$R = SF \cdot R_S = SF \cdot \frac{B}{2^{SF}}$ $R = SF \frac{B}{2^{SF}} \frac{4}{4 + CR}$	$E \propto \frac{SF}{2^{SF}}$ More ToA = More energy



Visualization of SF

These are **pseudo orthogonal signals**, this means that multiple signals with different SF can arrive concurrently,

since they have different data rates. In practice also on the same SF can be avoid collision if power difference is ≥ 6 dB. Moreover by nature this is a **pure ALOHA**.

LoRa also applies a Forward Error Correction (FEC) by adding a number $CR = \{1, 2, 3, 4\}$ of extra bits every 4 bits. Therefore a symbol $SF - CR$ information bits and the (information) bitrate is decreased (80% up to 50% of R_S).

Higher SF $\iff$ longer ToA $\implies$ We can accumulate energy for longer $\implies$ Better sensitivity and Longer Range. In fact the following table shows how a lower SNR is required for higher SF:

SF (bits/symbol)	SNR_m
7	-7.5 dB
8	-10 dB
9	-12.5 dB
10	-15 dB
11	-17.5 dB
12	-20 dB

SF vs SNR

While this shows a comprehensive list of all useful info, including Data Rates (DR) that go from 0 to 6.

SF	DR	Bandwidth (KHz)	Bit rate (Kbit/s) (CR = 1)	Range (km)	ToA (ms) (10-Byte)	Sensitivity (dBm)	Max. payload at app. (B)
7	DR6	250	10.9	2	28	-120.5	222
	DR5	125	5.5	2	56	-123	222
8	DR4	125	3.125	4	103	-126	222
	DR3	125	1.758	6	205	-129	115
10	DR2	125	0.977	8	371	-132	51
11	DR1	125	0.537	10	741	-134.5	51
12	DR0	125	0.293	12+	1483	-137	51

Full Table

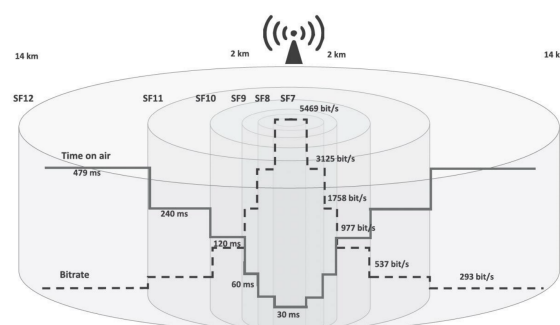
The sensitivity is calculated via

$$S[dBm] = -174 + 10\log(B) + NF + SNR_m$$

And notice that the max. allowed payload size as the application layer decreases as the SF increases in order to reduce the ToA (since it still naturally increases as SF increases).

ADAPTIVE DATA RATE

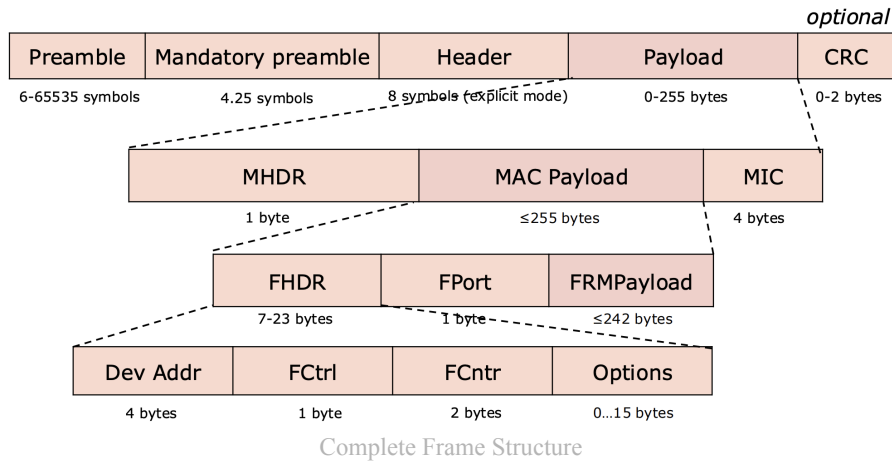
It is important to be able to automatically select the DR based on the conditions of the device. The SF should be high enough to reach the end device, but should avoid energy waste. The rule is: **as the distance of the end device from the gateway increases, the SF increases**



Adaptive Data Rate Visualaization

FRAME STRUCTURE

Here is the complete structure of the LoRa Frame



where:

- **Preamble:** Well known sequence of up-down chirps (usually 8 symbols)
- **Mandatory preamble:** for synchronization. 4.25 symbols means the last chirp is stopped after 1/4
- **Header:** Payload length and other useful info, in implicit mode it is omitted
- **CRC:** Cyclic Redundancy Check
- **MHDR:** MAC Header, specify message type
- **MAC Payload:** This has Min length of 12 +1 bytes as MHDR (1), Dev Addr (4), FCtrl (1), FCnt (2), MIC (4) + Fport(1)
- **MIC:** Message Integrity Code, digital signature
- **FPort:** Frame Port, defines app port number
- **FHDR:** Frame Header:
 - **Dev Addr:**
 - **FCtrl:** Frame Control: control flags, such as whether to use the data rate specified by the gateway for uplink transmission, whether this message acknowledges the reception of the previous message, whether the gateway has more data for the mote.
 - **FCntr:** Frame Counter, for security and replay protection
 - **Options:** Frame options, commands to change data rate, power consumptions etc

The entire ToA is the sum of all bits

$$ToA = (n_{preamble} + 4.25 + n_{payload})T_s$$

where $n_{preamble}$ are the symbols in the preamble and $n_{payload}$ are symbols in Payload+Header+CRC and can be calculated as:

$$n_{payload} = 8 + \max \left\{ \left\lceil \frac{8PL - 4SF + 28 + 16CRC - 20H}{4(SF - 2DE)} \right\rceil (CR + 4), 0 \right\}$$

where:

- 8: Min required payload
- 8PL: Mac Payload ≤ 255 B
- 4SF: Compensation factor as LoRa symbols don't carry linear number of information
- 28: internal padding and framing overhead
- 16CRC: If CRC is enabled it is 2 bytes=16 bits
- 4: LoRa interleaves at 4 bits per symbol
- SF: Each Symbol encoded in SF
- 2DE: Low Data Rate Optimization

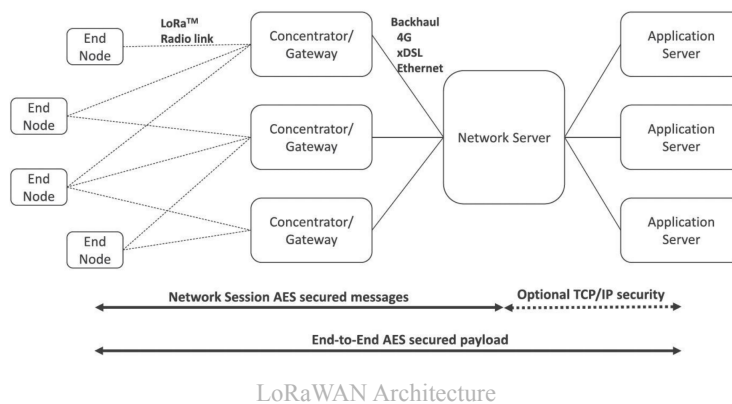
Moreover further constraints are applied, such as

- **1% Duty Cycle**

- **Fair Access Policy:** 30 seconds uplink time every day per device

TYPES OF END NODES

Now we dig into the LoRaWAN specific Layers



- **End Nodes:** carry sensing and actuating tasks (DevEUI)
- **Gateways:** Stars of start topology (DevAddr)
- **Network Server:** eliminates duplicate messages, routes data, determines the optimal data rate, selects the best Gateway to reach the End Node in DL

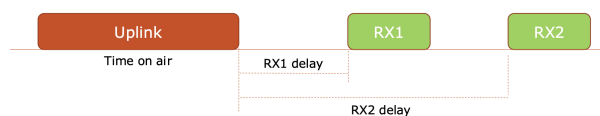
There are 3 types of classes of end nodes:

Class	A	B	C
Communication Type	Initiated by End Node	Bi-Directional	continuous receive
Pro	Very low energy consumption	Lower bounded DL latency	Very low DL latency
Cons	Very high DL latency	Higher Energy consumption	Very high power consumption

CLASS A

Communication always initiated by the End Node

End node sends an UL (uplink) message at an unspecified time (async). It then opens two RX windows for eventual DL. RX2 is opened only if there is no response during RX1.

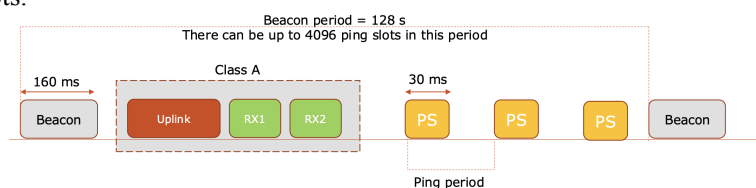


Class A

CLASS B

Communication is bi-directional

It incorporates class A but adds extra time synchronized beacons sent by the gateway to open ping slots. A beacon is sent every 128s and lasts for 160ms, a ping slot lasts 30ms. The interval between the two beacons (128 s) contains 4096 ping slots.

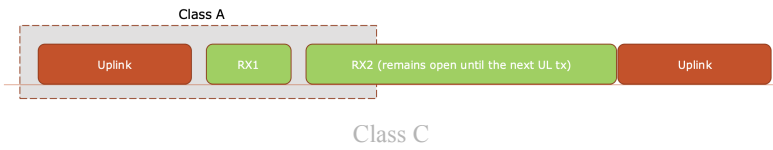


Class B

CLASS C

Continuous receive mode

RX2 remains open until next UL, unless there is activity on RX1



2) Cayenne LPP

Cayenne Low Power Payload (LPP) is a data formatter designed appositely for LoRaWAN. It defines a compact binary encoding/decoding for sensor readings.

First the Cayenne LPP library must be installed, then a script was provided by the professor (see [3\) Code, Lab 2](#)). After editing the code, the Cayenne LPP formatter must be chosen in TTN.

In order to retrieve this data from the TTN servers via an API, the API Key is required, this can be generated in the "Api Keys" section of TTN.

The Cayenne functions are the following:

```
//import and initialize
CayenneLPP lpp(51);
#include <CayenneLPP.h>

// LPP Packet commands
lpp.reset();

//Generic I/O
lpp.addDigitalInput(uint8_t channel, uint8_t value);
lpp.addDigitalOutput(uint8_t channel, uint8_t value);
lpp.addAnalogInput(uint8_t channel, float value);
lpp.addAnalogOutput(uint8_t channel, float value);

//Specific I/O
lpp.addLuminosity(uint8_t channel, uint18_t lux);
lpp.addPresence(uint8_t channel, uint8_t value);
lpp.addTemperature(uint8_t channel, float celsius);
lpp.addRelativeHumidity(uint8_t channel, float rh);
lpp.addBarometricPressure(uint8_t channel, float hpa);
lpp.addAccelerometer(uint8_t channel, float x, float y, float z);
lpp.addGPS(uint8_t channel, float latitude, float longitude, float meters);
```

Cayenne LPP Theory



Todo

3) BLE & MQTT

Bluetooth And Bluetooth Low Energy (BLE) Theory

Bluetooth is a short distance wireless technology. it has 3 classes:

- **Bluetooth Classic:** Basic Rate (BR) or Enhanced Rate (ER) used for file TX, audio streaming, peripheral devices
- **Bluetooth High Speed:** used for video steaming and tethering (hotspot)
- **Bluetooth Low Energy (BLE):** used for wearables, beacons, smart devices $\implies$ IoT

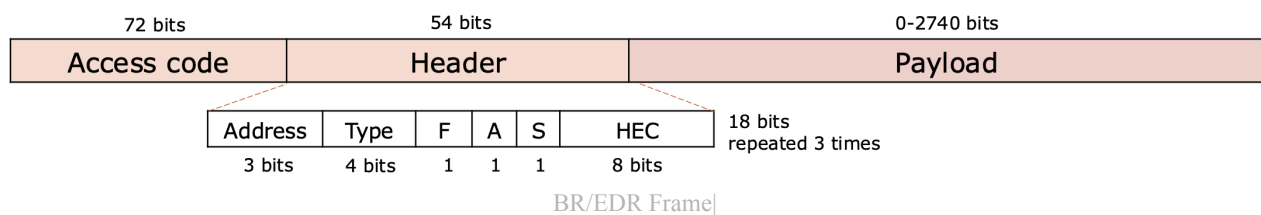
TOPOLOGY

Each BT network is called **piconet** and is a star topology with the master as the central node, the other devices are the slaves.

Up to 255 parked (in sleep) devices, but only 2-8 active devices simultaneously. The channels are accessed via polling.

FRAME

Now we analyze the frame of BT Classic BR/EDR:

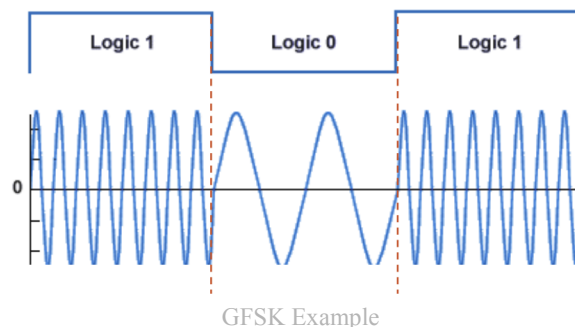


- **Access Code:** Sync bits and piconet ID
- **Header:** 18 bit patten repeated 3 times (coding)
 - **Address:** 3 bits = 7 possible destinations in piconet
 - **Type:** type of message
 - **F:** Flow (1=cannot accept packets)
 - **A:** Ack (S&W, 1 bit enough)
 - **S:** SeqNo (S&W, 1 bit enough)
 - **HEC:** Header Error Correction

RADIO LAYER (=PHY)

Three different modulation schemes:

- **Gaussian Frequency Shift Keying (GFSK) with gaussian filtering:** 1 Mbps. This modulation consists in shifting carrier frequency to represent 0 or 1. Gaussian filtering is used to filter out some of the interference



- **n/4-DQPSK:** 2Mbps
- **8-DQPSK:** 3 Mbps

It operates on the 2.4 GHz ISM band divided into 79 channels of 1 MHz each. Moreover it employs **Frequency-Hopping Spread Spectrum (FHSS)** where it changes the band 1600 times per second, it remains in the band for $625\mu s$ (**dwell time**). See [Baseband Layer](#) to understand how it changes bands.

The TX range depends on the TX power.

BASEBAND LAYER (=MAC)

It uses **Time Division Duplex (TDD)** with slots of 1 dwell time ($=625\mu s$). It consist in alternating the directions for UL and downlink: Master→Slave (even slots), Slave→Master (odd slots).

How do two devices connect?

- **Phase 1; Inquiry:** Master initializes communication link and generates a pseudo random hopping sequence for piconet then goes to sleep
- **Phase 2; Paging:** Master pages a slave. Slave replies with a Device Access Code (DAC), the master responds with the planned frequency hops (might take long). Slave sends another DAC to source

LOGICAL LINK CONTROL AND ADAPTATION PROTOCOL (L2CAP)

L2CAP is used for multiplexing, segmentation, reassembly, QoS and group management. It support two types of communications:

Synchronous Connection Oriented (SCO)	Asynchronous Connection-Less (ACL)
latency more important than integrity.	integrity more important than latency.
symmetric, synchronous	asymmetric, asynchronous
connection oriented	packet oriented
Mac is deterministic (slots reserved at regular time intervals)	Multislot (1, 3, 5), time intervals not regular

We also introduce the

- **Link Manager Protocol (LMP):** This is used to setup and manage Baseband connections
- **Host Controller Interface (HCI):** provides uniform command interface between LMP and Baseband layers
- **Service Discovery Protocol (SDP):** allows devices in a Bluetooth piconet to identify which services are offered by other devices and to learn their characteristics. Operates with request/response paradigm
- **Profiles:** specific tuning settings

BLUETOOTH LOW ENERGY (BLE)

BLE is a more recent extension aimed to low power devices.

- Only uses 40 (2 MHz) channels (vs 79 of 1 MHz)
- Only uses GFSK (2 Mbps) since higher data $\implies$ shorter ToA $\rightarrow$ less energy
- Fewer hops

BLE nodes are of 4 kinds:

- **Broadcaster:** a node which periodically transmits advertisements, but does not allow connections to be established (e.g., iBeacon).
- **Observer:** a node that just listens for advertisements and does not attempt to open connections (e.g., smartphone with an active localization App).
- **Peripheral (og Slave):** a node which transmits advertisements and may accept connection requests, acting as a Slave.

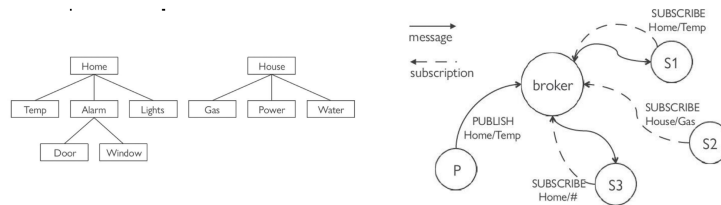
- **Central (og Master):** a node which may open connection towards a peripheral, acting as the Master once the information relative to a peripheral has been received through advertisements.

☑ GATT GAP and other stuff

Message Queuing Telemetry Transport (MQTT) Theory

MQTT is a IoT specific app protocol. It is layered on top of TCP or TLS and implements a publish-subscribe paradigm.

The base idea is the following: There is a central publisher that distributes messages to a broker. The broker forwards them to the subscribers. The subscribers never establish connections between each other. The subscriptions are made to specific topics.



MQTT Example

This protocol is lightweight, can be implemented on many platforms, has retain and will mechanism. However the centralized broker might create scalability issues.

FRAME STRUCTURE

MQTT messages are characterized by a minimal header for efficiency.

Fixed header (even though the length is not fixed)				
Packet type	Control flags	Remaining length	Variable header (optional)	Payload
4 bits	4 bits	1-4 bytes	N bytes	M bytes

Frame

- **Variable Header / Payload:** depends on message
- **Remaining Length:** represents sum of lengths of variable header and payload fields. Encoded using **Variable Byte Integer** which deploys a number of bytes based on represented int
- **Variable Byte Integer:** each byte encodes 7 bits of data and uses most significant bit as **continuation flag**. (1=more bytes, 0=no more bytes). Max is $2^{42} - 1$ bits ≈ 268 MB

Bytes	From	To
1	0	127
2	128	16'383
3	16'384	2,097,151
4	2,097,152	268,435,455

2 bytes	1	Data	Data	Data	Data	Data	Data	0	Data	Data	Data	Data	Data	Data
	More bytes to follow							NO more bytes to follow						

Overall, we have 14 data fields, so up to $2^{14} - 1 = 16'383$ B

Example

First we start with some basic definitions:

RETAINED MESSAGE

Since it is a client-server paradigm, subscribers cannot directly fetch data from publishers, but the latest data is obtainable through Retained Messages.

A message with the retained data field tells the broker to store the last message, on reconnection the subscribers will get that message.

For example , a data that rarely changes can be sent every T even if the data doesn't change. However, it is possible to send the data only when it changes under the form of a retained message in order to save battery of the publisher.

WILL MESSAGE

If a will message is enabled it will be published when the connection terminates ungracefully. It contains:

- Will Topic: where the broker will publish the message
- Will Message: the message payload

The will message can be sent as a retained message in order to let new subscribers know that the message was sent.

KEEP ALIVE

Since MQTT is based on TCP, we can have "half connection" problems, that is, connection disabled on one side but enabled on another; the data might be correctly sent but never reach the final destination.

The Keep Alive allows the MQTT server to determine if a half-connection problem arises and to then close the connection. Keep Alive is typically used in conjunction with Will Message: if the server does not receive any packets within the Keep Alive timer, it will send a Will Message.

Clearly the client must have $T < \text{Keep Alive}$.

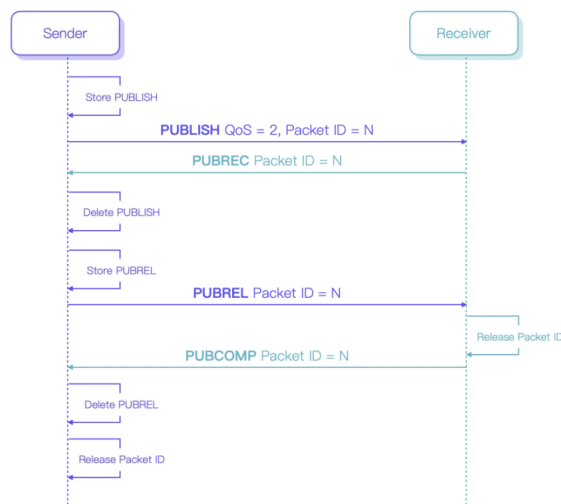
QoS LEVELS

QoS 0	QoS 1	QoS 2
At most once	At least once	Exactly once
The sender does not wait for acknowledgement or store and retransmit the message, so the receiver does not need to worry about receiving duplicate messages.	When the sender receives a PUBACK packet from the receiver, it considers the message delivered successfully. Until then, PUBLISH packet is stored for potential retransmission. Packet ID is used to match PUBLISH with PUBACK	The sender stores and sends a PUBLISH packet with QoS 2 and then waits for a PUBREC response packet from the receiver. This process is similar to QoS 1, with the exception that the response packet is PUBREC instead of PUBACK. The sender deletes it's copy The sender sends a PUBREL packet to release the Packet ID. It is stored for potential retx. The receiver responds with a PUBCOMP packet

Remark.

The effective QoS is

$$\text{Effective QoS} = \min \{\text{Publisher QoS}, \text{Subscriber QoS}\}$$



Example of QoS 2

Here are some examples from the terminal on how QoS is seen from a broker perspective

QoS=0

```

Received PUBLISH from pub1 (d0, q0, r0, m1, 'topic/test', ... (4 bytes))
Sending PUBLISH to sub1 (d0, q0, r0, m5, 'topic/test', ... (4 bytes))
  
```

QoS=1

```

Received PUBLISH from pub1 (d0, q2, r0, m1, 'topic/test', ... (4 bytes))
Sending PUBLISH to sub1 (d0, q2, r0, m5, 'topic/test', ... (4 bytes))

Sending PUBACK to pub1(m1, rc0)
Received PUBACK from sub1(Mid: 5, RC:0)
  
```

QoS=2

```

Received PUBLISH from pub1 (d0, q2, r0, m1, 'topic/test', ... (4 bytes))
Sending PUBREC to pub1 (m1, rc0)
Received PUBREL from pub1 (m1)
Sending PUBCOMP to pub1 (m1)

Sending PUBLISH to sub1 (d0, q2, r0, m5, 'topic/test', ... (4 bytes))
Received PUBREC from sub1 (Mid: 5, RC:0)
Sending PUBREL to sub1 (Mid: 5)
Received PUBCOMP from sub1 (Mid: 5)
  
```

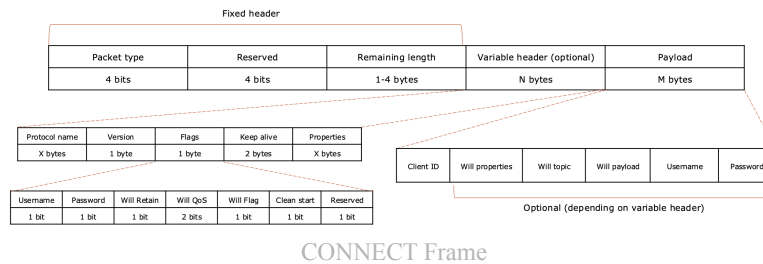
In synthesis, we can identify it based on the messages sent:

QoS Level	Handshake Packets in Log	Reliability	Speed
QoS 0	None (Just PUBLISH)	Lowest	Fastest
QoS 1	PUBACK	Medium (Guaranteed, maybe duplicates)	Medium
QoS 2	PUBREC, PUBREL, PUBCOMP	Highest (Guaranteed, unique)	Slowest

The **packet type** has many options, we just focus on: CONNECT, CONNACK, SUBSCRIBE, PUBLISH.

CONNECT

It deals with connections establishment, control and removal. CONNECT (re)opens a session iwth the broker and carries the ID of the client.



- **Username:** indicates whether payload contains username
- **Password:** indicates whether payload contains password
- **Will:**
 - **Retain:** indicates if will is retained
 - **QoS:** indicates QoS of message
 - **Flag:** indicates if payload contains flag
- **Clean Start:** indicates if session is new or continuation
- **Reserved:**
- **Keep Alive:** indicates time interval between two control packets sent

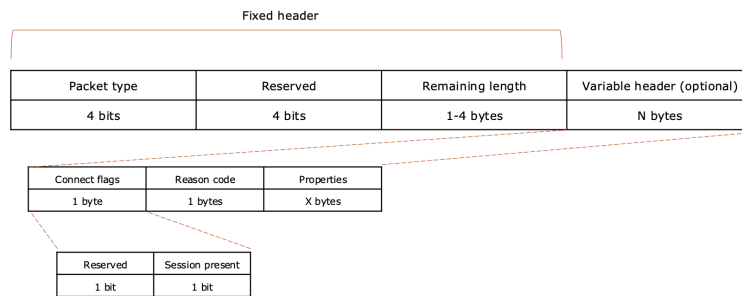
In this example a new connection was established with an username and password. It has a Will that is not retained and keep alive of 600s

CONNECT	
FLAGS	User Name Flag: 1
	Password Flag: 1
	Will Retain: 0
	Will QoS: 2
	Will Flag: 1
Clean Start: 1	
Keep Alive: 600	
ClientID: "S1"	
Will Topic: "Home/Temp"	
Will Payload: "S1 can't track Temp anymore"	
User Name: "users1"	
Password: "rmvt22"	

CONNECT Example

CONNACK

This is the response after a connect to inform the result. It has **no payload**

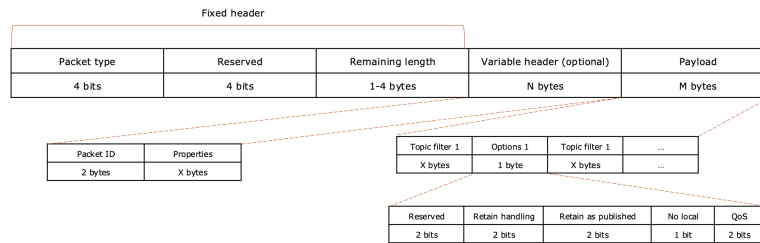


CONNACK Frame

- **Reason code:** indicates result of connection (both for good connections and failed ones)

SUBSCRIBE

It is used to initiate a subscription to a topic with a specified QoS level.

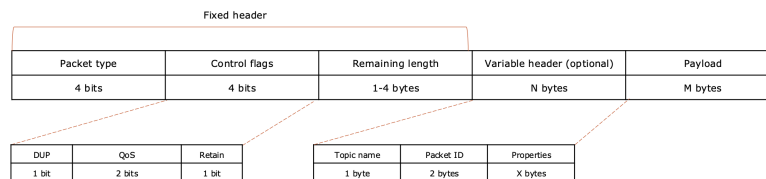


SUBSCRIBE Frame

- **Packet ID:** uniquely identify subscription request. PUBLISH, SUBSCRIBE, UNSUBSCRIBE use a set of identifiers
- **Retain Handling:** if the retained message should be received when the subscription happens
- **Retain as Published:** if the retain flag should be sent along the retained message
- **No Local:** Used to indicate whether the server can forward the application message to the publisher of the message
- **QoS:** max QoS level that the server can use

PUBLISH

This is used to publish the messages: either by client to server or server to publishers. It is followed by PUBACK, PUBREC, PUBREL and PUBCOMP.



PUBLISH Frame

- **DUP:** =1 if it is a RETX. The amount of ones received is indicative of the link quality
- **QoS:** Has priority
- **Retain:**
- **Topic Name:** what topic it should be published to
- **Payload:** The content of the message

4) Code

4) Lab 1

This code was used in the first lab. The first thing to do is to adjust the appEui and appKey at lines 33, 34

```
appEui = "8 HEX String";  
appKey = "16 HEX String";
```

This will first connect, then once per minute send a message (string) that will be visible in "Liv data"

```
// Before running edit properly line 42 and 43  
  
#include <MKRWAN.h>  
  
LoRaModem modem;  
  
String appEui;  
String appKey;  
String devAddr;  
String nwkSKey;  
String appSKey;  
  
String messages [4] = {"Welcome to IoT Lab", "This lab is amazing", "We will work  
with TTN" , "A few Arduino too <3"};  
int i = 0;  
  
void setup() {  
    int connected;  
    // put your setup code here, to run once:  
    Serial.begin(115200);  
    while (!Serial);  
  
    // change this to your regional band (eg. US915, AS923, ...)  
    if (!modem.begin(EU868)) {  
        Serial.println("Failed to start module");  
        while (1) {}  
    };  
  
    Serial.print("Your device EUI is: ");  
    Serial.println(modem.deviceEUI());  
  
    //edit with your own appEui and appKey  
    appEui = "A8610A3432207309";  
    appKey = "77A3076FAFE8D648A5853E77694FA5B8";  
  
    connected = modem.joinOTAA(appEui, appKey);  
  
    if (!connected) {  
        Serial.println("Something went wrong; are you indoor? Move near a window and  
retry");  
        while (1) {}  
    }  
    Serial.println("Succesfully Joined to the network!");  
}
```

```

void loop() {
  Serial.println("Waiting 1 minute to send next message");
  int j=0;
  while(j<120){
    delay(500);
    Serial.print("=");
    j++;
  }
  Serial.print("\n");
  while (modem.available()) {
    Serial.write(modem.read());
  }
  modem.poll();

  int err;
  modem.setPort(3);
  modem.beginPacket();
  modem.print(messages[i%4]);
  err = modem.endPacket(true);
  if (err > 0) {
    i = i+1;
    Serial.println("Message sent correctly!");
  } else {
    Serial.println("Error sending message :(");
  }
}

```

To decode these messages a simple javascript formatter was used

```

function decodeUplink(input) {
  return {
    data: {
      message: input.bytes.map(c => String.fromCharCode(c)).join('')
    },
    warnings: [],
    errors: []
  };
}

```

Lab 2

This code is used for Cayenne LPP. As in Lab 1 the correct appEUI and appKey are needed. This code sends a LPP packet that is defined in `void printVariables()`. The significant changes are:

```

#include <CayenneLPP.h> //added

...

CayenneLPP lpp(51); // instead of String messages [4] = {}; int i = 0;

...

void printVariables() // the LPP code is added here. Reset used to delete old buffer
  lpp.reset();

...

```

```
printVariables();
modem.write(lpp.getBuffer(), lpp.getSize()); //sends data
```

Recall that the correct payload formatter CayenneLPP must be chosen in TTN

```
// Before running edit properly line 42 and 43

#include <MKRWAN.h>
#include <CayenneLPP.h>

LoRaModem modem;

//edit with your own appEui and appKey
String appEui = "AAAAAAAAAAAAAAAA";
String appKey = "345C471D2F909810FBF6196B3A7A9D67";

CayenneLPP lpp(51);

void setup() {
  int connected;
  Serial.begin(115200);
  while (!Serial);

  if (!modem.begin(EU868)) {
    Serial.println("Failed to start module");
    while (1) {}
  };

  Serial.print("Your device EUI is: ");
  Serial.println(modem.deviceEUI());

  connected = modem.joinOTAA(appEui, appKey);

  if (!connected) {
    Serial.println("Something went wrong; are you indoor? Move near a window and
retry");
    while (1) {}
  }
  Serial.println("Succesfully Joined to the network!");
}

void wait(int seconds) {
  int j=0;
  while(j<seconds*2){
    delay(500);
    Serial.print("=");
    j++;
  }
  Serial.print("\n");
}

void printVariables() {
  lpp.reset();

  // Various Sensors
  float humidity = 40.0 + random(0,200)/20.0;
  lpp.addRelativeHumidity(3, humidity);

  float temp = 15 + random(0,2000)/100.0;
```

```

lpp.addTemperature(4, temp);

float pressure = 1013 + random(0,2000)/100.0;
lpp.addBarometricPressure(7, (pressure));

//this is wrong, too little precision
float ph = 6.99 + random(0,20)/1000.0;
lpp.addAnalogInput(8, ph);

//fixed with this, but the python data analyzer must be fixed
float ph = 10*(6.99 + random(0,20)/1000.0);
lpp.addAnalogInput(8, ph);
}

void loop() {
  Serial.println("Waiting 1 minute to send next message");
  wait(60);
  while (modem.available()) {
    Serial.write(modem.read());
  }
  modem.poll();

  int err;
  modem.setPort(3);
  modem.beginPacket();
  printVariables();

  // Debug: print payload
  Serial.print("Payload bytes: ");
  for (int i = 0; i < lpp.getSize(); i++) {
    Serial.print(lpp.getBuffer()[i], HEX);
    Serial.print(" ");
  }
  Serial.println();

  modem.write(lpp.getBuffer(), lpp.getSize());
  err = modem.endPacket(true);
  if (err > 0) {
    Serial.println("Message sent correctly!");
  } else {
    Serial.println("Error sending message :(");
  }
}
}

```

This is the data analyzer script. It's long, however we need to put our attention to two specific sections:

- The API Key, App EUI and Device EUI must be inserted. Also the "limit" (high int), and the "after" which is the starting date of the script (YYYY-MM-DDT:HH:MM:SSZ)

```

api_key = ""
application_id = ""
device_id = ""
limit = 0
after = "2020-10-10T10:10:10Z"

```

- The data analysis part is the following:

```

data = []
for d in uplink_data:
    try:
        dd = d["result"]["uplink_message"]["decoded_payload"]
        dd["time"] = d["result"]["uplink_message"]["received_at"]
        data.append(dd)
    except:
        pass
df = pd.DataFrame(data)
df["time"] = pd.to_datetime(df["time"])
df.sort_values("time", inplace=True)
df["analog_in_8"] = df["analog_in_8"].astype(float)/10.0

display(df)

```

Here is the full code

```

import requests
import json
import pandas as pd
from matplotlib import pyplot as plt
import seaborn as sns

def get_uplinks(api_key, application_id, device_id=None, limit=10, after="2020-08-20T00:00:00Z", field_mask=None):
    base_url = f"https://eu1.cloud.thethings.network/api/v3/as/applications/{application_id}/packages/storage/uplink_message"
    # If retrieving uplinks for a specific device, modify the URL
    if device_id:
        base_url = f"https://eu1.cloud.thethings.network/api/v3/as/applications/{application_id}/devices/{device_id}/packages/storage/uplink_message"
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Accept": "application/json"
    }
    params = {
        "limit": limit,
        "after": after
    }
    if field_mask:
        params["field_mask"] = field_mask
    response = requests.get(base_url, headers=headers, params=params)
    if response.status_code != 200:
        raise Exception(f"Error {response.status_code}: {response.text}")
    response = response.content.decode("utf-8")
    objs = []
    for line in response.split("\n"):
        if line.strip():
            objs.append(json.loads(line))
    return objs

api_key = ""
application_id = ""
device_id = ""
limit = 0
after = "2020-10-10T10:10:10Z"
field_mask = "up.uplink_message.decoded_payload"

```

```

uplink_data = get_uplinks(api_key, application_id, device_id, limit, after,
                           field_mask)

# Data Analysis
data = []
for d in uplink_data:
    try:
        dd = d["result"]["uplink_message"]["decoded_payload"]
        dd["time"] = d["result"]["uplink_message"]["received_at"]
        data.append(dd)
    except:
        pass
df = pd.DataFrame(data)
df["time"] = pd.to_datetime(df["time"])
df.sort_values("time", inplace=True)
#example how to resize data
df["analog_in_8"] = df["analog_in_8"].astype(float)/10.0

print(df.head())

# recall that <data_type>_<channel> is used for the specific names
df.rename(columns={"temperature_4": "Temperature (°C)",
                  "analog_in_8": "pH",
                  "barometric_pressure_7": "Pressure (hPa)",
                  "relative_humidity_3": "Humidity (%)",
                  "time": "Time"}, inplace=True)

df_melted = pd.melt(df, id_vars=["Time"], var_name="Sensor", value_name="Value")
g = sns.FacetGrid(df_melted, col="Sensor", col_wrap=1, sharex=True, sharey=False,
                  height=2.5, aspect=5)
g.map(sns.lineplot, "Time", "Value")
g.set_titles("")
g.set_xlabels("Time")
for ax, name in zip(g.axes.flat, g.col_names):
    ax.set_ylabel(name)
    ax.grid(True)
for ax in g.axes.flat:
    for label in ax.get_xticklabels():
        label.set_rotation(30)
        label.set_ha("right")
palette = sns.color_palette("tab10", n_colors=len(g.col_names))
for ax, color in zip(g.axes.flat, palette):
    lines = ax.get_lines()
    for line in lines:
        line.set_color(color)
plt.tight_layout()
plt.show()

```